

MTE 380 Milestone 3 Memo - Design Calculations Control System (Architecture + Fuzzy Scheduling)

Memo Summary

Purpose of document: detail the software architecture and control calculation.

Purpose of calculation: determine how sensor inputs (IR error and camera line visibility) are converted to adaptive speed limits and PID gains, then convert into bounded wheel commands for motor control.

Result of calculation: the controller computes normalized inputs X_1 and X_2 , evaluates fuzzy rules, and outputs mode-dependent $(v_{\max}, K_p, K_i, K_d)$ used in motor commands.

Final control output: left and right motor-drive outputs are computed from the PID-parameterized control law to drive the motors through the motor driver.

Design-decision support: this calculation supports the team's decision to use a state machine with adaptive control, instead of one fixed PID tuning for every mission phase.

1 Objective / Purpose Statement

Design objective: maximize line-following speed while maintaining stable tracking and preventing line loss.

Design constraints: limited camera lookahead, bounded motor commands, and phase-dependent mission tasks (rescue, drop-off, homing).

Design decision: use a state-machine architecture with fuzzy gain scheduling so each line-following phase can reuse the same control logic with adaptive parameters.

2 References

- G. Eleftheriou et al., "A Fuzzy Rule-Based Control System for Fast Line-Following Robots," *DCOSS 2020*, pp. 388–395, DOI: 10.1109/DCOSS49796.2020.00068.
- E. H. Mamdani, "Application of Fuzzy Algorithms for Control of Simple Dynamic Plant," *Proc. IEE*, 121(12), 1585–1588, 1974, DOI: 10.1049/piee.1974.0328.
- Pololu, QTR Reflectance Sensors documentation (readLine weighted-average position), accessed Feb. 2026.

3 Design Inputs and Specifications

3.1 Design Inputs

- $x_{j,k}$: commanded speed of motor j at control step k .
- $|X|$: maximum motor speed capability (rpm or normalized full-scale command).
- L_k : detected visible line length from camera ROI.

- $|L|$: maximum detectable line length in the selected ROI.
- e : lateral line-position error from IR sensor array.
- Δt : controller sampling period (s).

3.2 Design Specifications

Specification	Symbol / Parameter	Value / Range
State-machine phases	START to END	6 mission states + decision node
Fuzzy speed input	X_1	0 to 100
Fuzzy line input	X_2	0 to 100
Defuzzified output	X^*	1 to 100 (clamped)
Camera ROI (default)	roi_bottom_ratio	0.60
Row occupancy threshold	row_occupancy_frac	0.02
Motor command saturation	u_L, u_R	-1 to 1

4 Assumptions

- Camera segmentation in the bottom ROI is reliable under operating lighting.
- IR array provides a usable centroid error for PID correction in all line-following states.
- Commanded speed is used as a proxy for actual speed when wheel encoders are unavailable.
- Initial integral gain is set to $K_i = 0$ to reduce wind-up risk; tuning can be updated after testing.

5 Analytical Method and Computations

5.1 Architecture Method (State Machine)

Mission flow: START -> LINE_READING -> RESCUE -> GOTO_ZONE -> RESCUE_DEST_ARRIVED -> HOMING -> END.

- START: initialize sensors, control gains, and mission flags.
- LINE_READING: follow the line using IR error feedback plus camera-based fuzzy gain scheduling.
- Decision "line ended?": if false, remain in LINE_READING; if true, advance to rescue sequence.
- RESCUE: stop/slow as needed and enable pickup actuator (vacuum on).
- GOTO_ZONE: resume line-following control while carrying payload toward drop-off zone.
- RESCUE_DEST_ARRIVED: disable pickup actuator (vacuum off) and confirm drop-off completion.
- HOMING: follow return route with the same line controller under end-of-mission constraints.
- END: stop motors and terminate mission logic safely.

5.2 Underlying Engineering Principle for Controls

The underlying engineering principle is nonlinear closed-loop feedback control with sensor fusion and gain scheduling:

- **Closed-loop error regulation:** the IR array estimates lateral position error relative to line center, and PID control continuously reduces that error.
- **Gain scheduling:** controller aggressiveness is not fixed. It is adapted by fuzzy logic according to operating condition indicators (X_1 speed demand and X_2 visible line length).
- **Sensor fusion by role separation:** IR sensing provides high-rate local tracking error (e), while camera sensing provides lookahead visibility (L_k). Combining both improves robustness against either sensor's individual weaknesses.
- **Actuator-constrained control:** the computed commands are bounded and mapped to motor-driver voltage/PWM commands, enforcing physical limits and reducing saturation-induced instability.

5.3 Control Computation Mathematics

Step 1: architecture-level control decomposition (from paper):

$$x(k) = F_1(\hat{y}(k)) + F_2(\dot{y}(k))$$

where F_1 uses IR-array information for steering correction and F_2 uses camera lookahead information for speed/gain scheduling.

Step 2: IR-array feedback error (closed-loop variable):

$$e_k = \left(\frac{\sum_{i=1}^n i s_i}{\sum_{i=1}^n s_i} \right) - c$$

where s_i is the IR signal at sensor i and c is the center index. This error is the primary feedback variable regulated by PID.

Step 3: camera line-length calculation and normalization inputs:

Camera visibility is computed from ROI row occupancy:

$$L_k = \text{rows with line pixels in ROI}, \quad |L| = \text{ROI height in rows}$$

Normalized fuzzy inputs:

$$X_1 = \frac{\max_j x_{j,k}}{|X|} \times 100, \quad X_2 = \frac{L_k}{|L|} \times 100$$

Step 4: fuzzification model:

Crisp inputs (X_1, X_2) are mapped to graded memberships in $[0, 1]$. For standard triangular sets ($a < b < c$):

$$\mu_{\Delta}(x; a, b, c) = \begin{cases} 0, & x \leq a \text{ or } x \geq c \\ \frac{x-a}{b-a}, & a < x < b \\ \frac{c-x}{c-b}, & b \leq x < c \end{cases}$$

For shoulder sets (membership functions with a flat value of 1 on one side):

$$\mu_{\text{LS}}(x; a, c) = \max\left(0, \min\left(1, \frac{c-x}{c-a}\right)\right), \quad \mu_{\text{RS}}(x; a, c) = \max\left(0, \min\left(1, \frac{x-a}{c-a}\right)\right)$$

Input membership sets used by the scheduler are:

Variable	Label	Triangle (a, b, c)
X_1	Low	(0, 0, 40)
X_1	Medium	(20, 50, 80)
X_1	High	(60, 100, 100)
X_2	Close	(0, 0, 50)
X_2	Far	(30, 100, 100)

Apply shoulders where endpoints are repeated:

$$\mu_{X_1}^{\text{Low}} = \mu_{\text{LS}}(X_1; 0, 40), \mu_{X_1}^{\text{High}} = \mu_{\text{RS}}(X_1; 60, 100), \mu_{X_2}^{\text{Close}} = \mu_{\text{LS}}(X_2; 0, 50), \mu_{X_2}^{\text{Far}} = \mu_{\text{RS}}(X_2; 30, 100),$$

and use μ_{Δ} for interior sets (e.g., Medium).

Output labels represent control modes, each with its own output membership function:

$$\text{LC}(0, 10, 25), \text{LF}(15, 30, 45), \text{MC}(35, 50, 65), \text{MF}(55, 70, 85), \text{HC}(70, 85, 95), \text{HF}(85, 100, 100)$$

Use shoulder handling for repeated endpoints in output sets; specifically,

$$\mu_{\text{HF}}(x) = \mu_{\text{RS}}(x; 85, 100),$$

while LC, LF, MC, MF, and HC use μ_{Δ} . These output-label shapes are the targets used by the rule table in Step 5 and combined into one final value in Step 6. This step lets one input value belong partly to more than one label, so control changes are gradual instead of abrupt.

Step 5: rule base and firing strengths:

X_1	X_2	Output
Low	Close	LC
Low	Far	LF
Medium	Close	MC
Medium	Far	MF
High	Close	HC
High	Far	HF

The table is applied row by row. If row r has input labels (A_r, B_r) and output label L_r , then

$$\alpha_r = \min(\mu_{X_1}^{A_r}(X_1), \mu_{X_2}^{B_r}(X_2))$$

is the row strength for that exact row. For example, the row “Medium & Far $\rightarrow$ MF” gives

$$\alpha_{\text{MF}} = \min(\mu_{X_1}^{\text{Medium}}(X_1), \mu_{X_2}^{\text{Far}}(X_2)).$$

If multiple rows point to the same output label, keep the largest row strength for that label:

$$\mu_{\text{rule}}(L) = \max_{r \in L} \alpha_r$$

Step 6: aggregate outputs, defuzzify, and choose one label:

For each output label $L \in \{\text{LC}, \text{LF}, \text{MC}, \text{MF}, \text{HC}, \text{HF}\}$, let $\beta_L = \mu_{\text{rule}}(L)$.

$$\tilde{\mu}_L(x) = \min(\beta_L, \mu_L(x)), \quad \mu_{\text{agg}}(x) = \max_L \tilde{\mu}_L(x)$$

$$X^* = \frac{\sum_{x=0}^{100} \mu_{\text{agg}}(x) x}{\sum_{x=0}^{100} \mu_{\text{agg}}(x)}, \quad X_q^* = \min(100, \max(1, \text{round}(X^*, 2)))$$

$$L^* = \arg \min_{L \in \{\text{LC}, \text{LF}, \text{MC}, \text{MF}, \text{HC}, \text{HF}\}} |X_q^* - b_L|$$

where b_L is the peak value of label L . This step converts all fuzzy-rule outputs into one scheduler result and one control-mode label.

Step 7: map selected label to speed cap and PID gains:

$$(v_{\text{max}}, K_p, K_i, K_d) = \Theta(L^*), \quad v = \min(v_{\text{cmd}}, v_{\text{max}})$$

Label	$v_{\max}$	K_p	K_i	K_d
LC	0.30	0.80	0.00	0.10
LF	0.35	0.70	0.00	0.10
MC	0.45	0.65	0.00	0.12
MF	0.55	0.55	0.00	0.14
HC	0.65	0.45	0.00	0.16
HF	0.75	0.40	0.00	0.18

Step 8: compute PID correction and bounded wheel commands:

$$\begin{aligned}
I_k &= I_{k-1} + e_k \Delta t, & \dot{e}_k &= \frac{e_k - e_{k-1}}{\Delta t} \\
u_k &= K_p e_k + K_i I_k + K_d \dot{e}_k \\
u_{L,\text{raw}} &= v - u_k, & u_{R,\text{raw}} &= v + u_k \\
u_L &= \text{clamp}(u_{L,\text{raw}}, -1, 1), & u_R &= \text{clamp}(u_{R,\text{raw}}, -1, 1).
\end{aligned}$$

This converts scheduled gains and current error into safe, bounded wheel commands.

6 Calculations

6.1 Worked numerical example (one control cycle)

Assume one control cycle with

$$\begin{aligned}
x_{L,k} &= 0.62, \quad x_{R,k} = 0.68, \quad |X| = 1.00, \quad L_k = 78 \text{ rows}, \quad |L| = 120 \text{ rows}, \\
e_{k-1} &= 0.08, \quad e_k = 0.12, \quad \Delta t = 0.02 \text{ s}, \quad v_{\text{cmd}} = 0.70.
\end{aligned}$$

Normalized inputs:

$$X_1 = \frac{\max(0.62, 0.68)}{1.00} \times 100 = 68, \quad X_2 = \frac{78}{120} \times 100 = 65.$$

Non-zero memberships and corresponding non-zero rule strengths (all others are 0):

$$\begin{aligned}
\mu_{\text{Med}}(X_1) &= \frac{80 - 68}{80 - 50} = 0.40, & \mu_{\text{High}}(X_1) &= \frac{68 - 60}{100 - 60} = 0.20, & \mu_{\text{Far}}(X_2) &= \frac{65 - 30}{100 - 30} = 0.50, \\
\alpha_{\text{MF}} &= \min(0.40, 0.50) = 0.40, & \alpha_{\text{HF}} &= \min(0.20, 0.50) = 0.20. \\
X^* &= 75.30, & X_q^* &= 75.30, & L^* &= \arg \min_L |X_q^* - b_L| = \text{MF}.
\end{aligned}$$

Select MF parameters from Step 7:

$$v_{\max} = 0.55, \quad K_p = 0.55, \quad K_i = 0, \quad K_d = 0.14.$$

PID output and wheel commands:

$$\begin{aligned}
u_k &= 0.55(0.12) + 0 + 0.14 \frac{0.12 - 0.08}{0.02} = 0.346, \\
v &= \min(0.70, 0.55) = 0.55, & u_L &= \text{clamp}(0.55 - 0.346, -1, 1) = 0.204, \\
u_R &= \text{clamp}(0.55 + 0.346, -1, 1) = 0.896.
\end{aligned}$$

7 Results / Conclusions

The full calculation chain now provides a complete numeric path from sensed inputs to actuator commands: in the worked cycle, the scheduler selects MF, caps speed at $v_{\max} = 0.55$, and outputs $u_L = 0.204$ and $u_R = 0.896$ (both within the $[-1, 1]$ actuator bounds). This directly supports the objective of fast but stable line following and supports the design decision to use state-machine mission control with fuzzy gain scheduling, rather than one fixed PID tuning across all mission phases.